

Task 2 (Difficult): Real-Time Stream Analytics Engine

Problem Description

A cloud analytics platform receives millions of sensor readings. Each reading contains a sensor ID and temperature value. Using stream processing concepts, perform the following operations:

1. Filter temperatures greater than 50.
2. Group readings by sensor ID.
3. Compute average temperature per sensor.
4. Sort sensors based on average temperature in descending order.

Input Format

- First line contains integer N.
- Next N lines contain SensorID and Temperature.

Output Format

Display SensorID and average temperature sorted in descending order.

Constraints

- $1 \leq N \leq 10^5$
- $0 \leq \text{Temperature} \leq 100$

Sample Input

```
6
S1 60
S2 40
S1 80
S3 70
S2 90
S3 30
```

Sample Output

```
S1 70.0
S2 90.0
S3 70.0
```

Program:

```
package Taskss;

import java.io.BufferedReader;

import java.io.InputStreamReader;

import java.io.IOException;

import java.util.*;

import java.util.stream.Collectors;
```

```

class SensorReading {
    private final String id;
    private final double temperature;

    public SensorReading(String id, double temperature) {
        this.id = id;
        this.temperature = temperature;
    }

    public String getId() {
        return id;
    }

    public double getTemperature() {
        return temperature;
    }
}

public class task2 {
    public static void main(String[] args) throws IOException {
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        String firstLine = reader.readLine();
        if (firstLine == null || firstLine.trim().isEmpty()) return;
        int n = Integer.parseInt(firstLine.trim());
        List<SensorReading> readings = new ArrayList<>(n);
        for (int i = 0; i < n; i++) {
            String line = reader.readLine();
            if (line != null && !line.trim().isEmpty()) {
                String[] parts = line.trim().split("\\s+");
                readings.add(new SensorReading(parts[0], Double.parseDouble(parts[1])));
            }
        }
    }
}

```

```

    }
}

// Java 8 Stream Pipeline

readings.stream()

    // 1. Filter temperatures greater than 50

    .filter(r -> r.getTemperature() > 50)


    // 2 & 3. Group by Sensor ID and compute average temperature

    .collect(Collectors.groupingBy(

        SensorReading::getId,

        Collectors.averagingDouble(SensorReading::getTemperature)

    ))

    // Stream the map entries

    .entrySet().stream()

    // 4. Sort descending by average temperature (tie-breaker: ascending by Sensor ID)

    .sorted((e1, e2) -> {

        int cmp = Double.compare(e2.getValue(), e1.getValue()); // Descending order

        return (cmp != 0) ? cmp : e1.getKey().compareTo(e2.getKey()); // Secondary sort by ID

    })

    // Output result

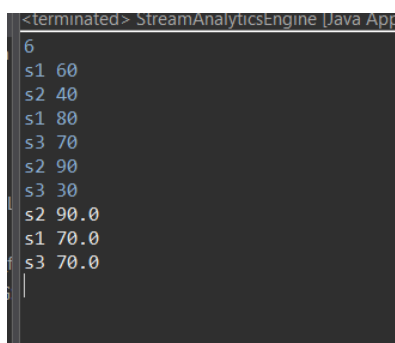
    .forEach(entry -> System.out.println(entry.getKey() + " " + String.format(Locale.US, "%.1f",
entry.getValue())));

}

}

```

Output:



```

<terminated> StreamAnalyticsEngine [Java App
6
s1 60
s2 40
s1 80
s3 70
s2 90
s3 30
s2 90.0
s1 70.0
s3 70.0

```